

MEGA CHEATSHEET

Tehnici de Programare — Java & OOP

Cursurile 1–11 • Toate Capcanele • Toate Conceptele

Cursul 1 — Ingineria Proceselor Software & Cerinte MoSCoW

1. Simptomele Codului Slab (Bad Code)

O grilă clasică de tip **multiple choice** cere să identifiți simptomele bad code. Cele 4 simptome majore:

Simptom	Descriere
Rigidity (Rigiditate)	Programul este greu de modificat — o schimbare afectează multe alte parti din cauza dependentelor proaste.
Fragility (Fragilitate)	Programul se strică în locuri ciudate/neașteptate când faci o modificare.
Coupled (Imobilitate)	Codul nu poate fi reutilizat (cannot be reused) în alte proiecte din cauza cuplării strânse.
Confusing	Codul este greu de citit și înțeles, necesitând code review constant.

2. Cerinte Functionale vs. Non-Functionale

Profesorii dau un scenariu și te pun să alegi tipul de cerință:

Tip Cerinta	Descriere & Exemple
Functionale	Descriu CE trebuie să facă sistemul. Ex: aplicatia permite unui angajat să adauge un produs, sau să vadă balanta contului.
Non-Functionale	Descriu CUM functionează sistemul — performanta, securitate, usabilitate. Ex: datele trebuie trimise criptat, dashboard-ul să se încarce în sub 2 secunde.

3. Tehnica MoSCoW (Prioritizarea Cerintelor)

Grilă clasică de tip **single choice**. Trebuie să știi exact ce înseamnă fiecare literă:

Litera	Semnificatie
M — Must have	Cerinte obligatorii. TREBUIE satisfăcute pentru ca solutia finală să fie un succes.
S — Should have	Prioritate înaltă, de dorit dacă este posibil. Apare frecvent în grile!
C — Could have	Cerinte dorite, dar nu absolut necesare (nice to have).
W — Won't have	Cerinte pe care stakeholderii au convenit că NU vor fi implementate acum (poate pe viitor).

4. Concepte Generale de Arhitectura

- ☐ **Problem Domain vs. Solution Domain:** Mapăm 'semantic gap'-ul dintre problema reală (Procese și Date) și soluție (Modele, Algoritmi, Programe) folosind abordarea OOP.
- ☐ **Top-Down:** Abordare procedurală (Divide et Impera clasic).
- ☐ **Bottom-Up:** Abordare Orientată pe Obiecte (OO).
- ☐ **Limbaje înalte:** Imperative (Procedural, OOP) și Declarative (Functional, Logic, DB Query). Stream-urile Java = stil declarativ!

Cursul 2 — UML Modeling & Arhitecturi Software

1. Tipuri de Diagrame UML

Categorie	Exemple de Diagrame
Structurale (Structural)	Class, Object, Component, Package, Deployment — arată arhitectura STATICĂ.
Comportamentale (Behavioral)	Use Case, Activity, State, Interaction — arată FLUXUL și INTERACȚIUNILE.

2. Relatii în Diagramele Use Case

- ☐ **<<extend>>:** Inserează un comportament OPTIONAL sau o excepție într-un use case de bază. Ex: Search face extend la 'Request a Book'.
- ☐ **<<include>>:** Un use case OBLIGATORIU folosește comportamentul altui use case. Ex: 'Place Order' include 'Login' — nu poți comanda fără login.
- ☐ **Generalizare:** Relatie de mostenire (părinte-copil) între use case-uri. Ex: Search generalizează Search by phone și Search by address.

3. Relatii între Clase (CEL MAI IMPORTANT — Punct MAJOR la Examen)

Trebuie să știi **diferențele fine** între toate tipurile de relatii:

Relatie	Descriere & Caracteristici Cheie
Dependency (Dependentă)	Cea mai slabă legătură. O clasă FOLOSESTE altă clasă ca parametru într-o metodă, nu o stochează.
Association (Asociere)	Relatie binară — un obiect contine o referință spre altul ca ATRIBUT. Poate fi unidirectionala, bidirectionala sau reflexivă (clasa se contine pe sine).
Aggregation (Agregare slabă)	Asociere de tip 'whole/part' (parte-întreg) sau 'has-a'. Este TRANZITIVĂ dar NU REFLEXIVĂ. Dacă întregul e distrus, PARTILE SUPRAVIETUIESC. Ex: dacă o filială bancară se închide, clientii continuă să existe.
Composition (Compozitie — Agregare strictă)	Agregare strictă. Durata de viață a părții COINCIDE cu întregul. Dacă banca dispare, dispar și filialele. Aceasta este STRONG AGGREGATION.
Realization	Implementarea unei interfete de către o clasă. Linie punctată cu săgeată goală.

⚠ CAPCANA CLASICA

Confuzia dintre Agregare și Compozitie este o capcana clasică. Diferenta cheie: în Compozitie, partile **NU** pot exista fără întreg (durata de viata coincidentă). În Agregare, partile supravietuiesc distrugerii întregului.

4. Vizibilitate UML

Simbol	Tip Vizibilitate
+	public
-	private
#	protected
~	package

Cursul 3 — Sabloane de Proiectare (Design Patterns)

Cel putin 2-3 grile din Design Patterns la fiecare examen. Sabloanele se împart în 3 categorii:

Categorie	Exemple Principale
Creational (Creare)	Singleton, Factory Method — ajută la instantierea obiectelor
Structural	Composite, Adapter, Proxy — compunerea claselor în structuri mai mari

Behavioral (Comportamentale)

Observer — definesc comunicarea și fluxul de control

1. Proxy (Structural)

- ☐ Utilizat pentru crearea unui REPRESENTANT (surogat) al unei operatii implementate pe o masina la distanță (remote machine).
- ☐ Sau: pentru a CONTROLA ACCESUL la un obiect.

⚠ CAPCANA PROXY vs ADAPTER

Dacă grila întreabă 'ce șablon folosești pentru o operație pe o masina la distanță?', răspunsul este PROXY, nu Adapter.

2. Adapter (Structural)

- ☐ **Rol:** Converteste interfata unei clase într-o altă interfață pe care o așteaptă clientul.
- ☐ Asigură integrarea versiunilor mai vechi ale codului (LEGACY) în aplicații OO noi.
- ☐ NU reprezintă ierarhii parte-întreg (acela e Composite)!

3. Composite (Structural)

- ☐ **Rol:** Reprezentarea ierarhiilor de tip PARTE-ÎNTREG (part-whole hierarchies).
- ☐ Îți permite să tratezi obiectele individuale și compozițiile de obiecte în MOD UNIFORM.
- ☐ Recunoști în cod: AbstractExpression, TerminalExpression, CompoundExpression unde CompoundExpression agregă AbstractExpression.

4. Factory Method (Creational)

- ☐ ASCUNDE logica de creare a instanțelor.
- ☐ **Recunoaște în cod:** O clasă (ex: ReportGenerator) cu o metodă care face `if(type.equals("txt")) return new TXTReport();`

5. Singleton (Creational)

- ☐ Asigură existența UNEI SINGURE INSTANȚE a unei clase.

Un Singleton **VALID** trebuie să aibă obligatoriu:

- **Constructor:** PRIVATE (nu public!)
- **Variabilă:** `private static NumClasa singleInstance;`
- **Metodă:** `public static NumClasa getInstance() { ... }`

⚠ CAPCANA SINGLETON

Constructorul public este motivul cel mai frecvent pentru care un Singleton din grilă NU respectă șablonul. Dacă constructorul este public, oricine poate crea mai multe instanțe!

6. Observer (Behavioral)

- ❑ Defineste o dependentă 1-la-n (1:n) între obiecte.
- ❑ Când un obiect (Subject) își schimbă starea, TOTI OBSERVATORII dependenti sunt notificati și actualizati AUTOMAT.

Cursul 4 — Multithreading & Concurenta în Java

1. Procese vs. Thread-uri

Concept	Detalii Cheie
Proces	Instanta unui program în execuție. Are propriul spațiu de adrese și resurse. Comunică prin IPC (Inter Process Communication).
Thread	Unitatea de execuție din interiorul unui proces. TOATE thread-urile unui proces împart același HEAP (memorie). Fiecare thread are propriul program counter, Stack și memorie privată.

2. Crearea Thread-urilor: extends Thread vs implements Runnable

Mod Creare	Avantaje / Dezavantaje
extends Thread	DEZAVANTAJ: clasa nu mai poate mosteni altceva (Java nu permite mostenire multiplă). Crează cuplare strânsă. Overhead suplimentar.
implements Runnable ✓ (RECOMANDAT)	AVANTAJ: clasa poate extinde altă clasă și implementa alte interfete. Cuplare slabă (loose coupling) — separă logica task-ului de managementul thread-ului.

3. wait() vs. sleep() — SUPER TESTAT

Proprietate	wait()
Apartine clasei	Object
Eliberează lock-ul?	DA — eliberează lock-ul pe obiect
Necesită synchronized?	DA — OBLIGATORIU din context sincronizat
Tip metodă	Metodă de INSTANTA

Proprietate	sleep()
Apartine clasei	Thread

Eliberează lock-ul?	NU — tine resursa blocată
Necesită synchronized?	NU — nu este necesar
Tip metodă	Metodă STATICĂ

4. Thread Safety

- ❑ **Variabile locale (primitive):** Stocate pe Stack-ul fiecărui thread. Nu sunt partajate => MEREU thread-safe.
- ❑ **Variabile instanța (atributele clasei):** Stocate pe Heap și partajate. NU sunt thread-safe fără sincronizare.
- ❑ **Obiecte Imutabile + Stateless:** Mereu thread-safe — dacă nu le poți modifica starea, mai multe thread-uri le pot citi simultan.

5. Sincronizare, Atomicitate & volatile

Atomicitate

- ❑ O operație este atomică dacă se execută complet sau deloc, fără a fi întreruptă.

⚠ CAPCANA CRITICA

Operațiile de tip read-modify-write (ex: `i++`, `count++`) NU SUNT ATOMIC! Ele constau în 3 pași: citire, modificare, scriere — un alt thread poate interveni între pași.

- ❑ Soluții: blocuri **synchronized** sau clase din **java.util.concurrent.atomic** (ex: `AtomicInteger`).

volatile

- ❑ Thread-urile copiază variabilele din RAM în Cache-ul CPU privat => probleme de VIZIBILITATE.
- ❑ Declarat **volatile**: forțează Java să citească/scrie DIRECT în RAM, ignorând cache-ul.

⚠ CAPCANA VOLATILE

volatile rezolvă VIZIBILITATEA, dar NU GARANTEAZĂ ATOMICITATEA pentru operații compuse (`i++`). Sunt două concepte diferite!

- ❑ **synchronized** previne accesul simultan la un bloc de cod. **CONSTRUCTORII NU POT fi declarati synchronized.**

6. Colectii Sincronizate vs. Concurente

Tip Colectie	Caracteristici
Sincronizate (<code>Collections.synchronizedList</code>)	Lock pe TOATA colectia pentru fiecare operație. CAPCANA: la iterare, programatorul TREBUIE să asigure manual sincronizarea: <code>synchronized(collection) { ... }</code>
Concurente (<code>ConcurrentHashMap</code> , <code>CopyOnWriteArrayList</code>)	Mai performante. <code>ConcurrentHashMap</code> împarte datele în segmente, permițând accesul simultan la segmente diferite.

7. Executor Framework & Virtual Threads

- ❑ **Thread Pools (Executors):** Separa crearea thread-urilor de aplicatie, reduc overhead-ul. Folosim `ExecutorService`.
- ❑ **Runnable:** returnează void. **Callable:** returnează rezultat ca Future, poate arunca exceptii.
- ❑ **Virtual Threads (Java 19 — Loom):** Thread-uri ultra-ușoare administrate de JVM (nu de OS). Nu blochează OS thread-ul la așteptare. Poti crea MILIOANE simultan cu overhead minim.

8. Probleme de Concurenta (Liveness Problems)

Problema	Descriere
Deadlock (Interblocare)	Două sau mai multe thread-uri blocate permanent — fiecare așteaptă ca celălalt să elibereze un lock.
Starvation (Înfometare)	Un thread nu primește niciodată timp de procesor — cauzat de thread-uri cu prioritate mare care acaparează resursele.

Cursul 5 — Tehnici de Design OOP

1. Încapsularea & Efecte Secundare

- ❑ **Mitul getter/setter:** Getter/setter NU garantează încapsulare perfectă! Pot expune structura internă. Adevărata încapsulare = modificările structurii interne NU afectează entitățile externe.
- ❑ **Functii pure:** Fără efecte secundare. Aceiasi parametri => acelasi rezultat. Sunt thread-safe.
- ❑ **Efecte secundare:** Orice modificare observabilă în afara metodei (atribut global, scriere fișier, exceptie). Programarea funcțională evită starea și favorizează imutabilitatea.

2. Legea lui Demeter (Law of Demeter / Principle of Least Knowledge)

Regula de aur — apare DES la grile: O metodă a unui obiect ar trebui să invoce DOAR metode ale:

1. Obiectului însuși (this)
2. Câmpurilor (atributelor) clasei respective
3. Parametrilor primiti de metodă
4. Obiectelor create LOCAL în interiorul acelei metode

⚠ INTERZIS — LEGE DEMETER

Chaining de gettere = INCALCARE GRAVA a Legii lui Demeter! Exemplu interzis:
`customer.getOrder().getPayment().process()` Acest cod accesează starea internă a obiectelor intermediare — o incalcare clara.

3. Mostenire (Inheritance) vs. Compozitie (Composition) — REGULA PRINCIPALA

Regula de aur: "Always Favor composition over inheritance!"

Criteriu	Mostenire (is-a) vs. Compozitie (has-a)
Tip reutilizare	Mostenire = White-box reuse (detalii interne vizibile, rupe encapsularea). Compozitie = Black-box reuse (buna encapsulare).
Cuplare	Mostenire = cuplare strânsă (tight coupling) între parinte și copil. Compozitie = mai putine dependente.
Flexibilitate	Mostenire = decizie STATICĂ (la compilare), nu se poate schimba la runtime. Compozitie = permite definirea dinamică la runtime prin polimorfism.
Dezavantaj compozitie	Cod mai lung, sisteme cu mai multe obiecte.

4. Regulile lui Coad (Când să folosesti TOTUSI Mostenirea)

Mostenirea e corectă DOAR DACĂ TOATE aceste 5 reguli sunt adevărate:

5. Subclasa exprimă o 'specie/un fel special de' și NU un rol jucat de superclasă.
6. O instanță a subclasei NU are nevoie niciodată să se transforme în obiect al altei clase.
7. Subclasa EXTINDE responsabilitățile, nu le suprascrie complet/anulează.
8. Subclasa nu extinde o clasă de utilitate.
9. Subclasa specializează o entitate/rol din Problem Domain.

⚠ EXEMPLU CLASIC GRESIT

Exemplu de incalcare (grilă): Person cu copiii Student și Passenger e un design gresit — un Passenger poate deveni Student în timp. Corect: Person compune un PersonRole.

5. Polimorfism & Inconsistente

- ☐ **Runtime Binding:** Decizia despre ce metodă să fie invocată se ia la executie, în functie de tipul REAL al obiectului.
- ☐ **Inconsistentă interfete:** Când o clasă implementează mai multe interfete cu suprapuneri (Student + Employee). Solutia: Simple Inheritance + Delegation.

6. Interfete vs. Clase Abstracte

Criteriu	Interfata vs. Clasa Abstractă
Interfata ('able / can do')	Describe ABILITATI periferice. Grupează clase eterogene fără legătură de natură (ex: Bicycle și Document implementează Recyclable). Recomandată când API-ul nu se va schimba des sau ai nevoie de mostenire multiplă.
Clasa Abstractă ('is-a')	Defineste IDENTITATEA de bază a descendentilor. Subclasele ei sunt omogene. Se foloseste pentru implementare de bază comună (common base class implementation).

Cursul 6 — Principiile SOLID & Inversion of Control

Principiile SOLID previn 'putrezirea design-ului' (Rotting Design) — momente în care codul devine rigid, fragil și greu de modificat.

1. Principiile SOLID

S — Single Responsibility Principle (SRP)

- ❑ O clasă ar trebui să aibă o SINGURĂ RESPONSABILITATE și un singur motiv pentru a fi modificată.
- ❑ **Recunosti incalcare:** O clasă care face și citire din fisier, și formatare de date, și procesare algoritmică. Schimbarea formatului fisierului te obligă să modifice clasa.
- ❑ **Solutia:** Spargi în clase mici specializate: FileReader, DataNormalizer, etc.

O — Open Closed Principle (OCP)

- ❑ Un modul ar trebui să fie DESCHIS pentru extensie, dar ÎNCHIS pentru modificare.
- ❑ **Recunosti incalcare:** O clasă cu tone de if/else if sau switch care verifică tipul obiectului. La adăugarea unui tip nou trebuie să spargi codul existent.
- ❑ **Solutia:** Polimorfism dinamic și interfete — se creează o interfață implementată de noile entități.

L — Liskov Substitution Principle (LSP)

- ❑ Subclasele trebuie să poată SUBSTITUI COMPLET clasele de baza fără să strice executia. Fiecare clasă promite doar ce poate face efectiv.
- ❑ **Recunosti incalcare:** O subclasă suprascrie o metodă a părintelui doar ca să arunce o excepție (ex: PremiumVideo.playRandomAd() aruncă excepție pentru că userii premium nu au reclame).

I — Interface Segregation Principle (ISP)

- ❑ Mai multe interfete specifice clientului sunt mai bune decât o singură interfață 'grasă' de uz general.
- ❑ **Recunosti incalcare:** Interfata LLMService cu metode pentru Text, Audio și Traducere. ChatBot e obligat să o implementeze, dar are nevoie doar de text => lasă restul metodelor goale sau aruncă UnsupportedOperationException.

D — Dependency Inversion Principle (DIP)

- ❑ Modulele de nivel înalt NU ar trebui să depindă de cele de nivel jos. Ambele ar trebui să depindă de ABSTRACTII (interfete/clase abstracte).
- ❑ **Regula din grile:** NU instantia (nu da new) clase concrete direct în cod. Depinde de interfața lor (ex: DatabaseAPI, nu MySQL_API).

2. Inversion of Control (IoC)

IoC inversează controlul standard al aplicației, făcând clasele slab cuplate, ușor de testat și de extins. Nu mai instantiezi manual dependentele.

Dependency Injection (DI)

- ❑ Clasa A nu mai face `b = new B()` — dependența e INJECTATA din exterior.
- ❑ Modalități: prin constructor (`public A(B b)`) sau prin metode setter/getter.
- ❑ **Grilă clasică:** 'O dependență este injectată când altă clasă C creează instanța lui B și o transmite lui A prin constructorul lui A, sau apelând un setter pe A.'

Service Locator (SL)

- ❑ Se ascund detaliile de obținere a dependenței. Clientul cheamă un 'Locator' (Broker) pentru a cere serviciul dorit.
- ❑ **Grilă:** Service Locator ESTE o tehnică IoC, dar presupune că clientul cheamă PROACTIV locatorul: `Locator.getService(...)`.

Callback Functions

- ❑ O funcție trimisă ca parametru altei funcții, destinată a fi executată după un eveniment (ex: click pe buton). Folosita intensiv în GUI.

3. Framework vs. Library vs. Design Pattern

Concept	Descriere & Regula
Library (Bibliotecă)	TU ești în control. TU chemi codul din librărie unde și când ai nevoie.
Framework	Inversează controlul programului tău. FRAMEWORK-UL îți cheamă codul ca răspuns la evenimente. (Principiul Hollywood: 'Don't call us, we'll call you')
Design Pattern	Soluții abstracte implementate manual de fiecare dată. Mai puține specializate decât un Framework.

Cursul 7 — Bune Practici OOP: Egalitate, Clonare & Imutabilitate

1. Egalitatea Obiectelor (equals)

Operator / Metodă	Ce Verifică
<code>== (operator)</code>	IDENTITATE — dacă două referințe indică spre EXACT ACELAȘI obiect din memorie.
<code>equals() (metodă)</code>	EGALITATE bazată pe STARE — dacă două obiecte diferite au aceleași valori.

Contractul metodei equals (FOARTE TESTAT)

10. **Reflexivitate:** `x.equals(x)` este MEREU true.
11. **Simetrie:** Dacă `x.equals(y)` este true, atunci și `y.equals(x)` trebuie true.
12. **Tranzitivitate:** Dacă `x.equals(y)` și `y.equals(z)` sunt true, atunci `x.equals(z)` trebuie true.
13. **Consistentă:** Apeluri multiple returnează același rezultat.
14. **Non-nulitate:** `x.equals(null)` trebuie să returneze FALSE.

⚠ CAPCANA CRITICA EQUALS

Capcana parametrului: Metoda corectă este public boolean `equals(Object o)` cu parametru de tip `Object`. Dacă declari public boolean `equals(MyClass o)`, NU suprascrii metoda din `Object` — faci OVERLOADING! Aceasta este o greșeală majoră și frecventă în grile.

⚠ MOSTENIRE SI EQUALS

Mostenire + equals = problemă! Nu există o modalitate simplă de a păstra contractul equals când extinzi o clasă instantiabilă și îi adaugi un atribut nou (ex: `ColorPoint` extinde `Point` și adaugă `color`). Orice încercare va încălca fie simetria, fie tranzitivitatea. SOLUTIA RECOMANDATA: favorizati compozitia în locul mostenirii!

2. hashCode

ℹ REGULA OBLIGATORIE

REGULA DE AUR: Ori de câte ori suprascrii `equals()`, ESTI OBLIGAT să suprascrii și `hashCode()`! Obiectele egale TREBUIE să aibă același hash code.

- ☐ Dacă `o1.equals(o2) = true` => `hashCode(o1) == hashCode(o2)` TREBUIE să fie adevărat.
- ☐ Dacă `o1.equals(o2) = false` => `hashCode` poate fi egal (se numeste COLIZIUNE). Dar performanța `HashMap`-ului e mai bună dacă hash-urile sunt diferite.

3. Clonarea Obiectelor (clone)

- ☐ **Conditie:** Clasa trebuie să implementeze interfața `Cloneable`.

⚠ MARKER INTERFACE

Interfața `Cloneable` este o INTERFATA MARCATOR (marker interface) — nu conține nicio metodă!

Contractul clone():

- ☐ `x.clone() != x` (obiect nou în memorie)
- ☐ `x.clone().getClass() == x.getClass()` (același tip)
- ☐ `x.clone().equals(x)` (starea copiată este identică)

Tip Copie	Descriere
Shallow Copy (Superficială)	Implementare DEFAULT a <code>super.clone()</code> . Copiază primitivele, dar pentru obiecte de tip referință copiază doar REFERINTA, nu obiectul. Originalul și clona pointează spre ACELEASI obiecte interne.
Deep Copy (Profundă)	Trebuie implementată MANUAL. Obiectele de tip referință sunt și ele clonate => structură complet independentă.

4. Clase Imutabile

Cum creezi o clasă imutabilă (cele 3 reguli):

15. Definesti attributele ca final.
16. NU pui metode setter (mutators).
17. Setezi totul DOAR prin constructor.

⚠ CAPCANA SUPREMA IMUTABILITATE

Capcana SUPREMA la imutabilitate: Simplul fapt că un atribut este declarat final NU face clasa imutabilă dacă atributul este o REFERINTA spre un obiect mutabil (ex: o listă, sau clasa Date). SOLUTIA: În metoda getter, returnează o CLONA a atributului, nu referinta originală: `return (Date) hireDate.clone();`

Cursul 8 — Reflection & Introspecție în Java

1. Concepte Fundamentale

- ☐ **Definiție:** Capacitatea unui program de a examina (introspecție) și modifica structura și comportamentul la RUNTIME, nu la compilare.
- ☐ **Punctul de intrare:** Un obiect de tip `java.lang.Class` — NU de tip Reflection, NU de tip Object!

Ce permite Reflection (toate sunt ADEVARATE în grile):

18. Crearea unei noi instanțe a unei clase în timpul execuției (`newInstance()`).
 19. Invocarea DINAMICĂ a metodelor în timpul execuției (`invoke()`).
 20. Accesarea metadatelor clasei (nume, pachet, metode, constructori).
 21. Accesarea și MODIFICAREA atributelor (chiar și cele private) ale unui obiect.
- ☐ **Dezavantaje:** Scade performanța, vulnerabilități de securitate, îngreunează mentenanța, face debugging dificil.

2. Cum obținem un obiect de tip Class

Situație	Metoda
----------	--------

Ai deja o instanță a obiectului	<code>myObject.getClass()</code> — metodă de INSTANTA (nu statică!)
Cunosti doar numele clasei ca String	<code>Class.forName("pachet.NumeClasa")</code> — metodă STATICĂ
⚠ CAPCANA GETCLASS O grilă falsă spune că poti obtine clasa folosind metoda STATICĂ <code>getClass()</code> din <code>Object</code>. FALS! <code>getClass()</code> este metodă de INSTANTA.	

3. Invocarea Dinamică a Metodelor — CAPCANA SUPREMA

Scenariu: Clasa `Operation` cu `public int sum(int a, int b)`. Cum invoci prin Reflection?

Pasul 1 — Obținerea metodei (`getMethod`)

- ☐ **Corect:** `Method sumMethod = aClass.getMethod("sum", new Class[] {int.class, int.class});`
- ☐ Primul argument = NUMELE metodei. Urmatorii = TIPURILE EXACTE ale parametrilor.

Pasul 2 — Invocarea metodei (`invoke`)

- ☐ **Corect:** `sumMethod.invoke(operation, new Object[] {4, 5});`
- ☐ Primul argument = INSTANTA obiectului pe care rulezi metoda. Urmatorii = VALORILE efective ale parametrilor.

⚠ GRESEALA FRECVENTA

Gresala frecventa în grile: `sumMethod.invoke(4, 5)` — LIPSESTE INSTANTA! Fara instanta ca prim argument, codul este incorect.

4. Accesarea Membrilor Privati

- ☐ Reflection permite ocolirea encapsulării (incalcarea OOP).
- ☐ **Pasii:** 1) `getDeclaredField("numeAtribut")` 2) `field.setAccessible(true)` — dezactivează verificarile de acces la runtime. 3) `field.get(obiect)` sau `field.set(obiect, valoare)`.

5. Adnotari (Annotations) — Restrictii Importante

Adnotarile (`@Table`, `@Column`) se folosesc cu Reflection pentru metadata (ex: mapare Hibernate/Spring).

Restrictii interfete adnotare (afirmatii ADEVARATE/FALSE la grile):

- O adnotare NU poate mosteni (extinde) altă adnotare.
- Metodele dintr-o adnotare NU pot avea parametri. (`String name();` corect, `String name(int x);` gresit)
- Metodele NU pot avea clauza `throws`.
- O adnotare NU poate fi generică (`@interface MyAnnotation<T>` este ilegal).

26. Tipul de return al metodelor este strict limitat la: **primitive, String, Class, enum, altă adnotare, sau array din oricare.**

Cursul 9 — Tipuri Generice & Principiul PECS

1. Concepte Fundamentale & Type Erasure

- ❑ **Scopul genericeilor:** Siguranța la compilare (compile-time type safety) și eliminarea cast-urilor explicite.
- ❑ **CÂND există genericele? CAPCANA:** Tipurile generice există DOAR la nivel de compilator (compile time)!

! TYPE ERASURE

TYPE ERASURE: Compilatorul verifică tipurile, apoi le STERGE (le transformă în tipuri raw sau Object) înainte de JVM. La runtime, JVM-ul NU știe nimic despre generice.

2. Invarianta (Subtipuri) — Cel mai testat concept teoretic

⚠ INVARIANTA — FALSA IN GRILE

Deși String este subtip al lui Object, List<String> NU ESTE subtip al lui List<Object>! Afirmatia 'List<String> este subtip al lui List<Object>' => FALS! Corect: 'List<String> nu este subtip al lui List<Object>', dar List<String> este subtip al lui List<String>'

3. Bounding (Limitarea Tipurilor Generice)

- ❑ Când definești o clasă/metodă cu tip generic limitat, folosești **extends** (ex: <T extends Comparable>), NICIODATĂ super!
- ❑ Un tip generic nelimitat este tradus automat de compilator ca Object.

4. Principiul PECS (Producer Extends, Consumer Super)

PECS te ajută să decizi ce wildcard (?) să folosești. Grilele îți dau fragmente de cod și te întreabă dacă sunt valide.

Tip Wildcard	Reguli PECS
? extends T (Upper Bound — PRODUCER)	Colectia produce (extragi/citești date din ea). Contine T și SUBTIPURILE sale. REGULA DE FIER: NU AI VOIE să adaugi niciun element! list.add(...) => EROARE DE COMPILARE.
? super T (Lower Bound — CONSUMER)	Colectia consumă (inserezi/scrii date în ea). Contine T și SUPERTIPURILE sale (până la Object). REGULA DE FIER: Când citești cu get(), primești Object, nu T. Integer n = ints.get(0) => EROARE, trebuie cast!

<T> (fără wildcard)	Dacă ai nevoie să si citesti si să scrii în aceeași colecție, NU folosi wildcard-uri!
---------------------	---

5. Cele 5 Restrictii Majore ale Genericelor (DES TESTATE!)

Toate generează erori de compilare:

27. **Nu poți instantia un tip generic direct:** `new T()` este INTERZIS — JVM-ul nu știe ce e T la runtime (Type Erasure).
28. **Nu poți folosi tipuri primitive:** `ArrayList<int>` => GRESIT. Trebuie `ArrayList<Integer>` (wrapper class).
29. **Clasele de excepții nu pot fi generice:** `public class MyException<T> extends Exception` => GRESIT. JVM-ul ar trebui să prindă la runtime informația stearsă.
30. **Nu poți crea array-uri de tipuri generice:** `E[] arr = new E[3]` => INTERZIS. (Permis: `Wrapper<?>[]` `arr = new Wrapper[3]` — wildcard nelimitat.)
31. **Nu există clase anonime generice:** Clasa anonimă nu are nume => nu poți specifica tipul parametrilor.

Cursul 10 — Lambda & Interfete Functionale

1. Programare Funcțională vs. OOP

Stil	Caracteristici
Functional (Declarativ)	Spui CE vrei să faci, lăsând biblioteca să rezolve detaliile. Ex: <code>cities.contains("Cluj")</code> .
OOP standard (Imperativ)	Folosești stări mutabile, bucle for, spui pas cu pas CUM să faci lucrurile.

- ☐ **Funcții pure:** Fără efecte secundare. Același input => același output. Nu modifică nicio stare globală.
- ☐ **High-order functions:** Iau alte funcții ca argumente SAU returnează funcții ca rezultat.

2. Expresiile Lambda

- ☐ **Definiție:** Un bloc de cod ANONIM. NU are: nume, tip de returnare declarat, clauza throws, sau tipuri generice proprii.
- ☐ **Cum inferă compilatorul tipul (IMPORTANT):** Tipul unei lambda este determinat de CONTEXT (target type). Acel tip TREBUIE să fie o Interfață Funcțională.

⚠ CAPCANA COMPILARE vs RUNTIME

Tipul parametrilor într-o expresie lambda implicită este determinat la COMPILARE (compile time), NU la execuție (runtime).

⚠ CAPCANA NUMAR PARAMETRI

Dacă interfata cere 2 parametri (ex: `add(double n1, double n2)`), dar declari lambda cu 3 parametri (`x,y,z`) -> `x+y+z` => EROARE DE COMPILARE! Numarul de parametri trebuie să se potriveasca exact.

- ❑ **Unde poate fi atribuită:** DOAR unei variabile de tip interfață funcțională, nu unui primitiv sau simplu obiect.

3. Interfete Functionale

! REGULA INTERFATA FUNCTIONALA

REGULA DE AUR: O interfață este funcțională dacă are EXACT O SINGURA METODA ABSTRACTA. O interfață poate contine oricâte metode default sau static, sau poate suprascrie metode din `Object` (`equals()`). Atâta timp cât are o singură metodă abstractă proprie, RAMINE FUNCTIONALA.

- ❑ **@FunctionalInterface:** Adnotarea este RECOMANDATA, dar nu obligatorie pentru ca o interfață să funcționeze ca atare.

4. Cele 4 Interfete Functionale de Baza (`java.util.function`)

Interfața	Semnatura & Metoda Abstractă & Metode Default
Function<T, R>	Primește T, returnează R. Metoda abstractă: <code>apply(T t)</code> . Default: <code>andThen()</code> și <code>compose()</code> pentru înlănțuire.
Predicate<T>	Evaluează o condiție. Primește T, returnează boolean. Metoda abstractă: <code>test(T t)</code> . Default: <code>and()</code> , <code>or()</code> , <code>negate()</code> .
Consumer<T>	Consumă un obiect. Primește T, NU returnează nimic (void). Produce side-effects. Metoda abstractă: <code>accept(T t)</code> .
Supplier<T>	Generează un obiect. NU primește argumente, returnează T. Metoda abstractă: <code>get()</code> .

5. Method References (Referinte la Metode)

- ❑ Referintele la metode (ex: `String::length` sau `System.out::println`) sunt o PRESCURTARE pentru expresii lambda.
- ❑ **Regula:** O referință la o metodă poate fi folosită DOAR acolo unde o expresie lambda poate fi folosită.

CE NU SUNT (grile false):

- ❑ NU sunt un pointer spre o funcție ca în C.
- ❑ NU sunt un tip nou de dată în Java.

⚠ CAPCANA EXECUTIE INTARZIATA

CAPCANA: Method references NU apelează metoda în momentul în care sunt declarate. Ele INDICA spre metodă; executia se face ulterior, când este chemată metoda din interfața funcțională.

Cursul 11 — Stream-uri & Iteratie Internă

1. Colectii vs. Stream-uri

Criteriu	Colectii vs. Stream-uri
Stocare	Colectia stochează EFECTIV toate elementele în memorie. Stream-ul NU are spatiu de stocare — procesează elemente ON-DEMAND dintr-o sursă.
Tip iteratie	Colectie = ITERATIE EXTERNĂ (programatorul scrie bucla: for-each, while(iterator.hasNext())). Stream = ITERATIE INTERNA (iteratia e realizată 'behind the scenes' de operatiile stream-ului).
Reutilizare (CAPCANA!)	Colectiile pot fi parcurse de câte ori vrei. Stream-urile NU POT FI REUTILIZATE! Dacă ai aplicat o operatie terminală, stream-ul se închide. Reutilizarea aruncă exceptie!

2. Tipuri de Operatii pe Stream-uri

! REGULA FUNDAMENTALA

Un stream este COMPLET INACTIV până când este invocată o operatie terminală!

Operatii Intermediare (LAZY / Leneșe)

- ❑ **Exemple:** filter, map, distinct, sorted, limit, skip
- ❑ Returnează MEREU un alt Stream. Nu execută nicio procesare în momentul scrierii — sunt amânate până la operatia terminală.
- ❑ **Loop Fusion:** Compilatorul combina un filter + map într-o SINGURA trecere (single pass), nu parcurge lista de două ori.
- ❑ **Grilă:** 'Operatia filter este de tip lazy' => ADEVĂRAT.

Operatii Terminale (EAGER / Nerăbdătoare)

- ❑ **Exemple:** forEach, reduce, collect, count, min, max, anyMatch
- ❑ Declanșează parcurgerea efectivă a stream-ului și produc un REZULTAT NON-STREAM (valoare primitivă, colectie sau void).

3. Operatia Terminala reduce

- ❑ Rolul ei: reduce tot stream-ul la o SINGURA VALOARE folosind un acumulator (ex: suma tuturor elementelor).

⚠ CAPCANA REDUCE

Capcana: 'Rezultatul operatiei reduce poate fi un stream.' => FALS! reduce returnează o valoare simplă (ex: double, int, un singur Obiect), NICIODATĂ un stream.

4. Crearea Stream-urilor (Secvente Infinite)

Metodă	Descriere
<code>Stream.iterate(seed, UnaryOperator)</code>	Pornește de la o valoare initială (seed) și aplică o funcție repetitiv. Ex: de la 2, obții toate numerele pare.
<code>Stream.generate(Supplier)</code>	Nu are nevoie de input inițial — generează valori noi mereu. Ex: <code>Math::random</code> .

⚠ STREAM INFINIT — LIMIT OBLIGATORIU
Aceste stream-uri sunt INFINITE! Trebuie OBLIGATORIU 'tăiate' cu operația intermediară `.limit(n)` înainte de operația terminală, altfel programul rulează la infinit!

5. Stream-uri Concurente (Parallel Processing)

- ❑ **Cum activezi:** `.parallelStream()` sau `.parallel()` — folosește intern framework-ul Fork/Join al Java.

Non-interference (Regula 1)

⚠ CONCURRENTMODIFICATIONEXCEPTION

Este INTERZIS să modifice sursa de date a stream-ului în timp ce iterezi! Dacă în interiorul stream-ului adaugi sau stergi ceva din colecția sursă => `CONCURRENTMODIFICATIONEXCEPTION`. SOLUȚIA: Folosește `CopyOnWriteArrayList` — creează o copie a array-ului la fiecare modificare.

Efecte Secundare (Regula 2)

- ❑ Modificarea unei variabile globale din afara stream-ului (în `forEach`) poate duce la rezultate imprevizibile (race conditions) dacă stream-ul rulează paralel.

Ordinea (Regula 3)

- ❑ Stream-urile paralele procesează pe fire diferite, deci **`forEach()`** standard poate printa elementele într-o ordine amestecată.
- ❑ **Soluția:** `forEachOrdered()` preservă ordinea inițială, dar reduce parțial avantajul de performanță al paralelismului.

Tabel Recapitulativ — Capcanele cele mai frecvente

Subiect	Regula de Memorat
Stream reutilizare	Stream-urile NU pot fi reutilizate după o operație terminală => excepție!
reduce	Rezultatul reduce este O VALOARE SIMPLĂ, niciodată un stream.

Invarianta generice	List<String> NU este subtip al lui List<Object>!
Type Erasure	Genericele există doar la compilare. JVM-ul nu le cunoaste.
new T() generice	INTERZIS — JVM-ul nu stie ce e T la runtime.
equals() parametru	public boolean equals(Object o) — parametru OBJECT, nu tipul clasei!
hashCode obligatoriu	Dacă suprascrii equals(), TREBUIE să suprascrii și hashCode().
final ≠ imutabil	final pe o referință nu face obiectul referit imutabil!
Cloneable	Interfată marcator (fără metode). Clasa trebuie să o implementeze pentru clone().
wait() vs sleep()	wait() = pe Object, eliberează lock. sleep() = pe Thread, NU eliberează lock.
i++ atomic?	NU! Este read-modify-write (3 pasi). Foloseste AtomicInteger sau synchronized.
volatile	Rezolvă VIZIBILITATEA, dar NU ATOMICITATEA operatiilor compuse.
Singleton	Constructor PRIVATE + variabilă private static + metodă publică statică getInstance().
Proxy vs Adapter	Proxy = acces la distantă/control acces. Adapter = convertește interfețe, integrează legacy.
Legea lui Demeter	Interzis: customer.getOrder().getPayment().process() (chaining de gettere).
Runnable vs Thread	Runnable = recomandat (loose coupling, permite mostenire altceva). Thread = cuplare strânsă.
@FunctionalInterface	Recomandata, NU obligatorie. O interfată e functională cu EXACT 1 metodă abstractă.
Method Reference executie	NU apelează metoda la declarare — executia e amânată.
Framework vs Library	Library: TU chemi codul. Framework: EL îți cheamă codul.
filter lazy/eager?	filter = operatie LAZY (intermediară).
forEach lazy/eager?	forEach = operatie EAGER (terminală).
Reflection intrare	Punctul de intrare = java.lang.Class, NU Object, NU Reflection.
invoke() Reflection	Primul argument = INSTANTA obiectului, nu valorile parametrilor!

Succes la examen! 